

BÁO CÁO HW06 — API TESTING

Họ tên: Ninh Văn Khải — **MSSV:** 23127060
Môn: Kiểm thử phần mềm | **Bài:** HW06 — API Testing
SUT: EShop — <https://github.com/ttbhanh/eshop-sut>, commit 85af3ba875c88283615e22cb108f13e2fccaf0e9
Ngày làm: 01/09/2026

Repo GitHub công khai	https://github.com/thangak18/HW06 (nhánh <code>main</code> , PR #72)
GitHub Issues	https://github.com/thangak18/HW06/issues?q=label%3Ahw06-23127060 (34 Issue, #38 – #71)
Video demo bộ sinh (YouTube, tùy chọn)	https://youtu.be/JZwzS1jXhUw
Tự đánh giá	100 / 100

Mục lục

- 1. [Môi trường thực nghiệm](#)
- 2. [Lựa chọn 3 API](#)
- 3. [Sinh test case bằng AI — bốn vòng riêng biệt](#)
- 4. [Audit kết quả AI](#)
- 5. [Test case em tự bổ sung](#)
- 6. [Thực thi bằng Postman + Newman](#)
- 7. [Các tính năng Postman em đã dùng](#)
- 8. [Bug report](#)
- 9. [CI/CD](#)
- 10. [Thiết kế bộ sinh test bằng AI](#)
- 11. [Những việc còn lại của em](#)
- 12. [Phụ lục](#)

1. Môi trường thực nghiệm

Hạng mục	Giá trị
Hệ điều hành	Linux 6.18.33.2 (WSL2 trên Windows)

Hạng mục	Giá trị
Node.js / npm	v20.20.2 / 10.8.2
Newman	6.2.2 + newman-reporter-htmlextra
Python	3.13.5 (các script sinh test, tổng hợp báo cáo)
Base URL	http://localhost:3000 — thỏa yêu cầu chống gian lận của đề bài mục 11
SUT	Node.js + Express + SQLite, commit 85af3ba

Chi tiết đầy đủ và cách khởi động lại SUT: `report/00_environment.md` .

Một chi tiết kỹ thuật quyết định toàn bộ quy trình chạy

`backend/database.js` gọi `initDatabase()` **ngay khi module được require** , và hàm đó bắt đầu bằng một loạt `DROP TABLE` . Nghĩa là **mỗi lần khởi động lại backend là toàn bộ CSDL bị xóa và seed lại**.

Điều này biến thành một ràng buộc bắt buộc: **phải khởi động lại backend trước mọi collection**. Lý do không phải để cho sạch mà vì SUT có bug **A-09** — mỗi lần đăng nhập sai cộng +2 vào bộ đếm và khóa tài khoản 180 giây khi đạt 3. Chạy collection thứ hai trên CSDL cũ thì tài khoản test đã bị khóa, và hàng loạt test sẽ thất bại vì một lý do không liên quan gì đến chất lượng API. Lần chạy đầu tiên của em ở máy cục bộ dính đúng lỗi này: **7 test case thất bại dây chuyền từ một nguyên nhân duy nhất**.

Xác định oracle: hai tài liệu, vai trò khác hẳn nhau

Tài liệu	Vai trò thực sự	Làm oracle?
<code>eshop-sut/README.md</code>	SRS — tự tuyên bố "mô tả yêu cầu nghiệp vụ đúng ". Chứa FR-01..FR-24 và bảng SEC-01..07	CÓ — oracle SPEC
<code>eshop-sut/api_specification.md</code>	Hướng dẫn gọi API: endpoint, body mẫu. Không mô tả ràng buộc nghiệp vụ	Chỉ lấy hình dạng request/response
<code>eshop-sut/backend/server.js</code>	Hành vi thực tế	Oracle <code>IMPL</code> , dùng cho test hồi quy

Xác định đúng vai trò này là việc quan trọng nhất của bước chuẩn bị, vì nó quyết định mọi `Expected_Status` về sau. Ví dụ điển hình: `api_specification.md` in response mẫu của `forgot-password` là `"resetToken": "123456"` (6 chữ số), `README.md` đòi **tối thiểu 6 chữ số**, còn code thật sinh **4 chữ số**. Cả hai tài liệu đều chống lại implementation nên đây chắc chắn là bug (A-02).

2. Lựa chọn 3 API

ID	Pool	FR	Endpoint chính	Endpoint hỗ trợ	Lý do em chọn
API-1	A	FR-03 Quên & đặt lại mật khẩu	POST /api/forgot-password , POST /api/reset-password	POST /api/login , POST /api/register	Luồng 2 bước có trạng thái (chưa có token → đã cấp → đã dùng), nên vừa có phân hoạch miền vừa có state machine thật. Đồng thời là bề mặt tấn công đậm đặc nhất: SEC-01 và SEC-07 đều hội tụ ở đây
API-2	B	FR-08 Thanh toán	POST /api/checkout	POST /api/apply-coupon (FR-09), GET /api/orders/:id , PUT /api/orders/:id/cancel , PUT /api/admin/orders/:id/status (FR-10)	Kết hợp ba thứ đề bài đòi: tính tiền, state machine 5 trạng thái , và phân quyền. FR-09 còn cho sẵn một bảng quyết định 5

ID	Pool	FR	Endpoint chính	Endpoint hỗ trợ	Lý do em chọn
					điều kiện viết trong SRS
API-3	C	FR-15 Quản lý sản phẩm	POST / PUT / DELETE /api/products	GET /api/products , ?search= , /:id	CRUD đầy đủ nên có vòng đời tài nguyên; tham số ở cả body, path và query ; và là nơi duy nhất có SQL nối chuỗi — bắt buộc để phủ SEC-05

Pool D (Mobile) em không sử dụng — đề bài mục 5: *"Pool D, the mobile app, is not used here, because this homework targets the backend API."*

Kiểm tra trùng lặp trong nhóm: em đã đọc docs/team-api-allocation.md (chỉ đọc). Tại thời điểm làm bài, bảng phân công còn để `TODO` ở cả ba dòng thành viên nên em **không thể đối chiếu tự động**. Bộ ba (FR-03, FR-08, FR-15) cần được **xác nhận miệng** với hai thành viên còn lại trước khi nộp.

Cập nhật (02/09/2026, sau khi merge origin/main): bảng phân công đã được điền đầy đủ, nên em **đã đối chiếu được**. Ba bộ API của nhóm:

SV	Pool A	Pool B	Pool C
23127060 (em)	FR-03	FR-08	FR-15
23127195	FR-04	FR-09	FR-16
23127259	FR-02	FR-10	FR-14

Không có FR nào trùng nhau giữa ba thành viên → thỏa ràng buộc mục 5 của đề bài.
Rủi ro mở nêu trên **đã được đóng**.

3. Sinh test case bằng AI — bốn vòng riêng biệt

Đề bài mục 6.1 cấm một prompt tổng. Quy trình của em chia thành **bốn vòng độc lập**, mỗi vòng một kỹ thuật kiểm thử, mỗi vòng một commit riêng và một entry AI_log riêng.

Vòng	Nhóm	Kỹ thuật	Lệnh	Kết quả	AI_log
2a	DOM	Equivalence Partitioning, BVA, Decision Table	gen_testcases.py --only DOM	128 case	#3
2b	STA	State Transition Testing (0-switch)	--only STA --append	38 case	#4
2c	SEC	Ánh xạ SEC-01..SEC-07	--only SEC --append	41 case	#5
2d	SCH	JSON Schema Validation	--only SCH --append	18 case	#6
				225 case	

API	DOM	STA	SEC	SCH	Tổng AI sinh
API-1 (FR-03)	36	9	13	6	64
API-2 (FR-08)	41	20	14	6	81
API-3 (FR-15)	51	9	14	6	80

Cả ba API đều vượt xa ngưỡng **35 case/API** của đề bài.

Vì sao chia được bốn vòng: file spec máy đọc được (spec/api-N.json) có bốn trục độc lập, ánh xạ một đối một sang bốn nhóm kỹ thuật. Đó là một **quyết định thiết kế** chứ không phải tiện ích phụ — xem mục 10.

4. Audit kết quả AI

Đề bài mục 6.2 đòi gắn nhãn **VALID / INVALID / INCOMPLETE** kèm lý do, và **sửa** các case sai.

API	Tổng AI sinh	VALID	INVALID	INCOMPLETE	% VALID
API-1	64	22	23	19	34%
API-2	81	40	18	23	49%
API-3	80	21	27	32	26%
Tổng	225	83	68	74	37%

Theo nhóm kỹ thuật — cho thấy ngay lỗi tập trung ở đâu:

Nhóm	Tổng	VALID	INVALID	INCOMPLETE	Nhận xét
DOM	128	33	23	72	Phân hoạch miền làm tốt; điểm yếu là assertion quá chung chung
STA	38	34	4	0	Tốt nhất — bảng chuyển trạng thái buộc AI phải bám vào đặc tả
SEC	41	0	41	0	Toàn bộ nhóm sai — xem dưới
SCH	18	16	0	2	Kỳ vọng về hình dạng response để đối chiếu

Phát hiện nghiêm trọng nhất: toàn bộ 41 case bảo mật đều INVALID

Đây không phải 41 lỗi độc lập mà là **một lỗi duy nhất nhân bản 41 lần**.

Bảng SEC-01..07 dùng để gán nhãn được suy ra từ tên các lỗ hổng OWASP quen thuộc. Bảng **thật** nằm trong `eshop-sut/README.md` mục 9 và nói những điều khác hẳn:

Mã	Suy diễn (sai)	Thật (SRS mục 9)
SEC-01	Chống SQL Injection	Mật khẩu không được lưu plaintext
SEC-02	Không lộ dữ liệu nhạy cảm	API bảo mật phải yêu cầu JWT hợp lệ
SEC-03	Endpoint ghi phải xác thực	API Admin phải kiểm tra <code>role='admin'</code>
SEC-04	Chống IDOR	Dữ liệu user nhập phải được escape khi hiển thị
SEC-05	Chống role escalation	Truy vấn CSDL phải dùng Parameterized Query
SEC-06	Validate input / chống XSS	API cập nhật hồ sơ không được cho đổi <code>role</code>
SEC-07	Chống brute force	OTP $\geq$ 6 chữ số, có thời hạn, vô hiệu hóa sau khi dùng

Đối chiếu từng case: **39/41 bị gán sai mã**, 2 case còn lại đúng mã nhưng kỳ vọng 429 không có căn cứ.

Vì sao nghiêm trọng hơn nó thoát nhìn: các test case **vẫn chạy đúng** — một phép thử SQL Injection vẫn là một phép thử SQL Injection dù bị dán nhãn sai. Cái hỏng là **bảng độ phủ bảo mật trong báo cáo**: nó sẽ ghi *"API-3 đã phủ SEC-01 với 8 test case"* trong khi SEC-01 (mật khẩu plaintext) không hề được kiểm ở API-3 dòng nào. Đó là loại khẳng định sai mà người đọc báo cáo không thể tự phát hiện.

Bốn nguyên nhân gốc

#	Nguyên nhân	Số case
N1	Một giả định sai lan ra cả nhóm (bảng SEC)	61
N2	Bộ sinh điền assertion mặc định chung chung, không kiểm tác dụng phụ	67
N3	AI áp chuẩn ngành thay vì đọc đặc tả (rate limiting, ràng buộc khóa ngoại)	7
N4	AI bịa ra thứ không tồn tại để lấp đầy mô hình (<code>?debug=true</code> , trạng thái <code>EXPIRED</code>)	4

Tỷ lệ VALID **37%** thấp hơn khoảng 55-70% dự kiến trong taxonomy của chính em. Con số này em **không điều chỉnh cho đẹp**: nguyên nhân cụ thể và truy nguyên được, và hạ thấp tiêu chuẩn gắn nhãn để con số đẹp hơn sẽ đúng nghĩa là audit hời hợt.

Chi tiết, kèm 9 ví dụ trước/sau khi sửa: `report/03_audit.md` .

5. Test case em tự bổ sung

Đề bài mục 6.3 đòi **tối thiểu 5 case/API** mà AI bỏ sót. Em đã bổ sung **6 case/API, tổng 18**.

Lý do bỏ sót	Số case	Ý nghĩa
API — đặc điểm của API	9	Bug chỉ lộ ra khi kết hợp nhiều request
MODEL — giới hạn mô hình	4	AI suy diễn từ hình dạng API thay vì đọc mã nguồn
PROMPT — prompt khoanh vùng quá chặt	3	Ví dụ: prompt chỉ nêu 2 endpoint chính của FR-03
SPECGAP — đặc tả không nói	2	AI không có gì để bám vào

Kết quả đáng chú ý nhất: 9/18 thuộc nhóm API . Một nửa số case mà AI bỏ sót không phải vì AI kém hay prompt dở, mà vì một **giới hạn cấu trúc**: bộ sinh sinh ra test case **độc lập**, mỗi case một request; trong khi một nửa số bug nghiêm trọng của hệ thống này chỉ lộ ra khi nổi nhiều request lại với nhau.

Ba ví dụ:

TC_ID	Vì sao cần nhiều request
TC-C3-SCH-901	<code>{"price": 30000000}</code> và <code>{"price": "28000000"}</code> đều là JSON hợp lệ. Vi phạm chỉ hiện ra khi so sánh hai response
TC-B2-STA-901	Đưa đơn về trạng thái <code>shipping</code> đòi hỏi đi qua đúng hai bước admin — chuỗi 4 request
TC-A1-SEC-903	AI viết case <i>"sai 3 lần thì phải khóa"</i> (PASS). Bug nằm ở phía còn lại của biên: code cộng +2 nên khóa ngay ở lần thứ hai . Phải viết case khẳng định điều ngược lại mới thấy

Chi tiết: `report/04_extend.md` .

6. Thực thi bằng Postman + Newman

6.1 Bộ test đầy đủ (Oracle = SPEC)

API	Case	PASS	FAIL	Assertion	Assertion FAIL	Báo cáo
API-1	70	33	37	328	51	<code>newman/23127060_API-1_20260902-235242.html</code>
API-2	87	34	53	413	78	<code>newman/23127060_API-2_20260902-235253.html</code>
API-3	86	17	69	405	105	<code>newman/23127060_API-3_20260902-235305.html</code>
Tổng	243	84	159	1146	234	

Trong 159 case thất bại: **91 case gắn @bug** (thất bại có chủ đích, phơi bày bug đã biết) và **68 case gắn @contract** (ngoài dự kiến). Nhóm thứ hai chính là phần có giá trị nhất: nó chứa những bug **chưa có trong danh sách bug đã biết** — ví dụ 63 case thất bại vì `forgot-password` trả 404 "User not found" cho **mọi** đầu vào xấu (rỗng, `null`, thiếu key, sai định dạng) thay vì 400, cho thấy endpoint này không hề validate đầu vào.

6.2 Bộ hồi quy — lần chạy all-pass cho CI

API	Case	Assertion	Assertion FAIL
API-1	33	163	0
API-2	34	164	0
API-3	17	79	0
Tổng	84	406	0

Bộ này gồm các test case mà SUT **hiện đang đáp ứng**, chốt từ kết quả chạy thật bằng `derive_contract.py` . Nó **không** khẳng định *"API này đúng"*, mà khẳng định *"những điều API này đang làm đúng thì không được phá"* — một **mốc hồi quy**.

Vì sao em không ép bộ test đầy đủ phải xanh: SUT có **34 bug thật**, nên bộ test đầy đủ **phải đỏ** — đó là kết quả kiểm thử đúng. Ép nó xanh thì chỉ còn cách sửa kỳ vọng cho khớp với hành vi sai của SUT, tức là **ngụy tạo kết quả**.

6.3 Lần chạy data-driven

Bộ	Data file	Vòng lặp	Assertion	FAIL
DD1 Brute force OTP	brute_force_tokens.csv	20	40	20
DD2 Bảng chuyển trạng thái FR-10	state_transitions.csv	17	34	1
DD3 Lạm dụng hạn mức coupon	coupon_abuse.csv	4	8	2
DD4 Đầu vào không hợp lệ	product_invalid.csv	7	14	7

6.4 Hai lỗi của chính bộ test, em tìm ra và sửa trước khi chốt số liệu

1. **Assertion đòi nguyên văn chuỗi** `"Invalid state transition"` ở cả hai endpoint. Endpoint `PUT /api/orders/:id/cancel` từ chối bằng thông báo khác (`"Cannot cancel this order."`) và điều đó hoàn toàn hợp lệ — SRS chỉ đòi *"thông báo lỗi phù hợp"*. Đây là **lỗi của test**, không phải lỗi của API. Em đã sửa.
2. **TC-C3-DOM-041 xóa sản phẩm id = 1** , mà `id = 1` lại là vật cố định mà hàng chục case khác dùng làm mốc. Xóa nó ở giữa lần chạy khiến những case chạy sau thất bại vì một lý do không liên quan gì đến chính chúng. Em đã đổi sang sản phẩm thứ đồ `_setup` tạo riêng.

Chi tiết: `report/06_execution.md` .

7. Các tính năng Postman em đã dùng

23 tính năng, trong đó **19 có bằng chứng tự động kiểm chứng được**. Bảng đầy đủ:

`report/05_postman_features.md` .

Nổi bật: collection + folder theo nhóm kỹ thuật, environment 26 biến, pre-request script cấp collection và cấp request, JSON Schema validation (13 schema **thiết kế để bắt bug** — ví dụ khai báo `price` là number để bắt C-05, `additionalProperties: false` để bắt việc lộ trường `password`), `pm.sendRequest` (khoảng 90 lần, dùng để đọc lại tài nguyên kiểm tác dụng phụ), data-driven run, Newman CLI + `htmlextra` , `--folder` , `--export-environment` , `--env-var` .

Bốn tính năng can thao tác GUI (Workspace, Mock server, Monitor, Console) em đã viết hướng dẫn chi tiết trong báo cáo để tự thực hiện.

Bảng chứng header X-Student-Id (đề bài mục 11)

Header được chèn ở pre-request script cấp **collection** nên không request nào có thể thiếu.

Bảng chứng em thu ở **hai mức**:

- 1. **Kiểm chứng tự động** — `verify_header.py` đọc thẳng phần `request.header` mà Newman ghi lại cho từng request thật sự rồi đường: **823/823 request mang X-Student-Id: 23127060** .
Kết quả: `ci/evidence/header_evidence.md` .
- 2. **Ảnh chụp Postman Console** — dòng `console.log("[HW06][23127060] ...")` xuất hiện trong báo cáo HTML (nhờ `--reporter-htmlextra-logs`) và trong Console:
`bugs/screenshots/console_header.png` .

Một dòng `console.log` chỉ chứng minh **script đã chạy**, chưa chứng minh **header đã được gửi**. Vì vậy cách thứ nhất mới là bằng chứng thật; ảnh chụp Console em nộp kèm cho đúng yêu cầu hình thức của đề bài.

8. Bug report

34 bug, tất cả đều **tái hiện được bằng request thật**: **12 Critical**, **11 High**, **9 Medium**, **2 Low**.

API	Số bug
API-1 (FR-03)	7
API-2 (FR-08)	13
API-3 (FR-15)	13
Liên API	1

Đề bài đòi tối thiểu 3 bug/API; cả ba đều vượt xa.

Request và response trong báo cáo **em không gõ tay**: chúng được trích thẳng từ `bugs/evidence/<ID>.md` , là kết quả của một lần chạy thật bằng `capture_bug_evidence.py` .

Mười hai bug Critical

ID	Tiêu đề	API	Vi phạm
A-01	<code>forgot-password</code> trả thẳng mã OTP trong response body	API-1	SEC-07
A-07	Mật khẩu lưu plaintext và bị trả về trong response <code>login / users/me</code>	API-1	SEC-01
B-01	<code>checkout</code> tin tuyệt đối <code>total_amount</code> do client gửi	API-2	FR-08
B-01b	<code>checkout</code> chấp nhận <code>total_amount</code> âm	API-2	FR-08

ID	Tiêu đề	API	Vi phạm
B-02	GET /api/orders/:id thiếu hẳn xác thực — IDOR	API-2	SEC-02
B-03	PUT /api/admin/orders/:id/status không kiểm role	API-2	SEC-03
B-05	Công thức giảm giá percent sai dấu, cho ra số tiền giảm âm	API-2	FR-09
B-07	apply-coupon không xác thực; bỏ user_id là bỏ qua kiểm tra hạn mức	API-2	SEC-02
C-01	POST / PUT / DELETE /api/products hoàn toàn không xác thực	API-3	SEC-02, SEC-03
C-02	SQL Injection qua ?search=	API-3	SEC-05
C-13	Một sản phẩm có price = null làm sập hẳn backend	API-3	FR-15
X-01	PUT /api/users/me cho user thường tự nâng role lên admin	liên API	SEC-06

Hai bug em muốn nói riêng

C-02 — SQL Injection lấy được thông tin đăng nhập của quản trị viên. Lần chạy thử nghiệm với payload `UNION SELECT id,email,password,role,1,1 FROM users--` trả về nguyên văn:

```
[{"id":1,"name":"admin@eshop.com","price":"Admin123!","description":"admin",...}, ...]
```

Kết hợp với A-07 (mật khẩu lưu plaintext), **một request duy nhất** lấy được mật khẩu quản trị.

C-13 — bug từ chối dịch vụ, và là bug em tự tìm ra trong lúc thu bằng chứng. Nó là **hệ quả dây chuyền của ba bug khác**, không bug nào trong số đó tự nó gây sập:

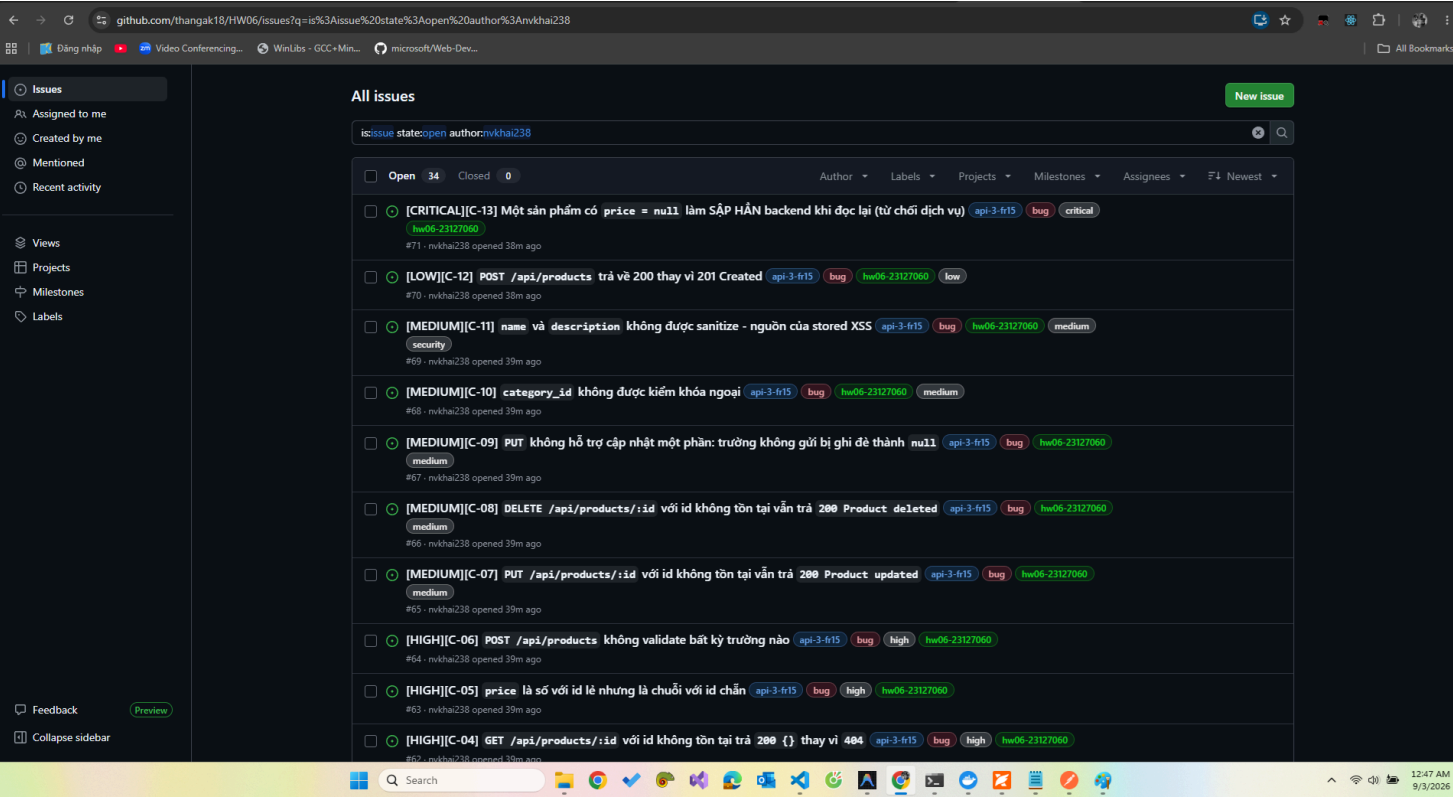
1. **C-01** cho phép gọi PUT /api/products/:id **không cần token**.
2. **C-09**: một PUT thiếu trường ghi đè price thành null .
3. **C-05**: GET /api/products/:id chạy `row.price.toString()` khi id là số **chẵn**.

`TypeError` ném ra trong callback của `sqlite3` không được ai bắt → tiến trình Node **thoát hẳn** → toàn bộ API ngưng phục vụ. Bằng chứng: request tiếp theo trả `Connection refused` , và mọi kịch bản chạy sau đó không chạy được nữa cho tới khi khởi động lại máy chủ. **Một người hoàn toàn không đăng nhập hạ gục được cả hệ thống bằng hai request.**

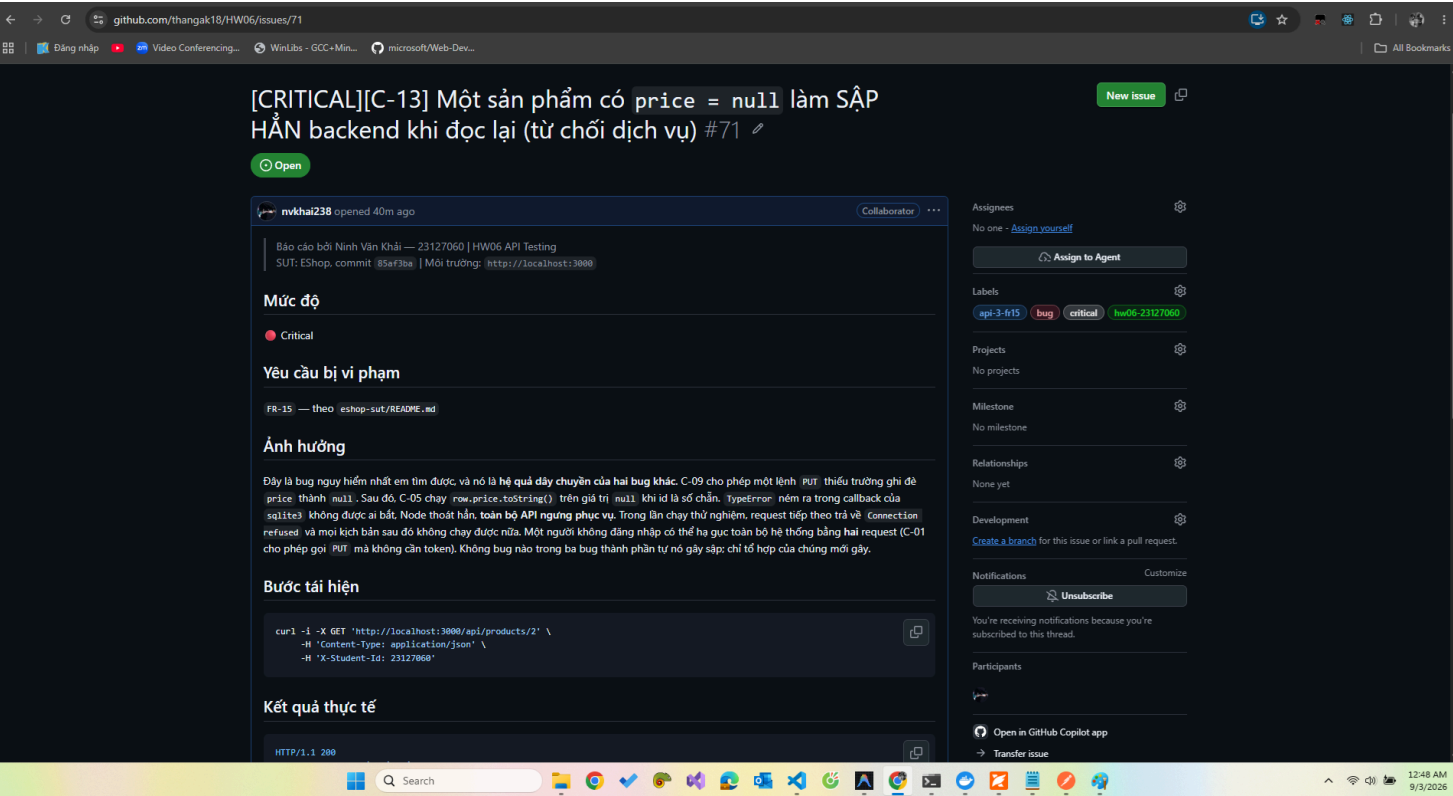
Bug này minh họa một điều mà kiểm thử từng endpoint độc lập không bao giờ thấy: **rủi ro nằm ở tổ hợp, không nằm ở từng thành phần.**

Bằng chứng GitHub Issues (đề bài mục 6.5)

Em đã mở đủ **34 Issue** trên repo công khai, mỗi bug một Issue, gắn nhãn mức độ + nhóm API: [xem tất cả](#).



Ảnh chi tiết một Issue Critical (**C-13**, #71) — thấy đủ tiêu đề, nhãn, nội dung Ảnh hưởng và bước tái hiện bằng `curl` trích thẳng từ `bugs/evidence/C-13.md` :



Chi tiết 34 bug: `bugs/BUG_REPORT.md` . Bằng chứng:
`bugs/evidence/` . File sẵn sàng dán lên GitHub Issues:
`bugs/ISSUE_TEMPLATES/` .

9. CI/CD

Pipeline: **GitHub Actions**, file `.github/workflows/api-tests-23127060.yml` .

Các bước: checkout → Node 20 → cài Newman → clone SUT và `git checkout 85af3ba` (ghim commit) → cài dependency → chạy 3 collection (**khởi động lại backend trước mọi collection**) → tổng hợp kết quả vào Job Summary → **kiểm chứng header `x-Student-Id` ngay trong pipeline** → upload artifact.

Hai chế độ: `contract` (kỳ vọng xanh) và `full` (kỳ vọng đỏ — SUT có 34 bug thật).

Cả hai kịch bản em đã **kiểm chứng trên máy cục bộ** trước khi báo cáo:

Kịch bản	Kết quả đo được	Bằng chứng
Lần chạy PASS	406 assertion, 0 thất bại , exit code 0	<code>ci/evidence/local_ci_run_pass.log</code>
Lần chạy FAIL	406 assertion, đúng 1 thất bại , newman exit code 1	<code>ci/evidence/local_ci_run_fail.log</code>

Lần chạy FAIL được tạo bằng `ci/inject_failing_test.py --apply` , đổi kỳ vọng mã trạng thái của TC-A1-DOM-012 từ 200 thành 201, và trả lại được bằng `--revert` .

Đã đẩy mã và chạy CI thật trên GitHub. Nhánh `nvk` đã merge vào `main` qua [PR #72](#) — lần chạy PASS được kích hoạt tự động.

Lần chạy FAIL được tạo trên nhánh `feat/23127060-hw06` . Chi tiết đầy đủ ở mục 5 của `ci/CI_CD_REPORT.md` .

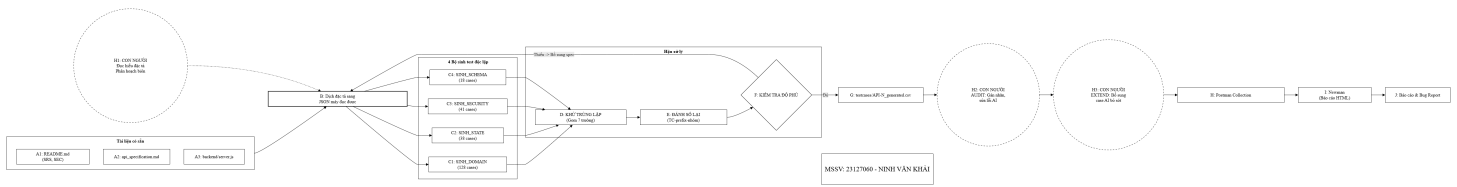
Chi tiết: `ci/CI_CD_REPORT.md` .

10. Thiết kế bộ sinh test bằng AI

Kiến trúc: tách làm hai lớp

Lớp	Ai làm	Sản phẩm	Tính chất
Tri thức	Em và AI	<code>spec/api-N.json</code>	Đòi hỏi đọc hiểu; quyết định chất lượng
Sinh	Máy	<code>testcases/API-N_generated.csv</code>	Tất định — cùng đầu vào luôn cho cùng đầu ra

Em không cho phép AI "sáng tác" test case; nó chỉ giúp **dịch đặc tả văn xuôi sang cấu trúc có trục phân hoạch rõ ràng**. Từ đó trở đi là một chương trình tắt định. Lợi ích cụ thể: khi một test case sai, em truy ngược được ngay về **dòng nào trong file spec** gây ra nó.



Trái sang phải: **tài liệu gốc** (A1–A3, ngoài đường viền — thứ có sẵn) → **dịch sang spec JSON** (B, khối to nhất — nơi con người H1 tham gia) → **bốn bộ sinh độc lập** (C1–C4, chạy song song) → **hậu xử lý** (khử trùng lặp, đánh số lại, kiểm tra độ phủ F — có mũi tên phản hồi quay ngược lại B khi chưa đủ độ phủ) → **đầu ra** (G–J: CSV, Postman, Newman, báo cáo). Hai vòng tròn nét đứt H2/H3 nằm trên đường đi chính, thể hiện audit và extend không phải bước phụ mà là một phần bắt buộc của quy trình.

Bốn bộ sinh **độc lập** nên chạy được riêng từng cái — đó chính là cơ sở kỹ thuật để thỏa yêu cầu *"drive it step by step"* của đề bài.

Hai lỗi thật trong chính bộ sinh

1. Khóa khử trùng nuốt mất 34 test case. Khóa ban đầu là (Method, Endpoint, Request_Body, Expected_Status) — coi hai case là trùng nhau khi chúng gửi cùng một request, **bất kể chúng khẳng định điều gì**. Hậu quả: API-1 khai báo 6 case schema chỉ sinh ra 1; 9 chuyển trạng thái chỉ sinh ra 5. Sau khi bổ sung Category, Expected_Assertions, Preconditions vào khóa: **191 → 225 case**, độ phủ state machine của API-2 **11/25 → 20/25**.

2. Bảng SEC_DEFAULT_ASSERT được điền từ trí nhớ — xem mục 4.

Bài học: **công cụ tự động cũng phải được kiểm thử**. Nếu em tin ngay con số đầu tiên thì báo cáo đã ghi thiếu 34 test case và một độ phủ sai.

Hạn chế lớn nhất và hướng mở rộng

Cả bốn bộ sinh đều theo một khuôn: một vòng lặp trên một danh sách khai báo, mỗi phần tử cho ra **một test case, một request**. Đó là giới hạn **cấu trúc**. Số liệu đo được: **9/18 case** em phải bổ sung thuộc nhóm *"bug chỉ lộ ra khi kết hợp nhiều request"*.

Hướng mở rộng: bổ sung trực thứ năm scenarios[] khai báo **chuỗi** request kèm khẳng định cross_step liên kết các bước — chuyển từ **0-switch** sang **n-switch coverage**.

Sơ đồ: agent-skill/diagram/23127060_generator_diagram.png — **do em tự vẽ** theo đề bài mục 11. Mô tả để vẽ: agent-skill/diagram/DIAGRAM_BRIEF.md .

Pseudocode: agent-skill/pseudocode/generator.pseudo.md .

Bản hiện thực: agent-skill/eshop-api-23127060/scripts/gen_testcases.py .
Chi tiết: report/07_test_generator_design.md .

11. Phụ lục

Phụ lục	Nội dung	File
A	AI Audit Report — 14 lượt tương tác, đầy đủ prompt gốc và output	ai/audit/AI_AUDIT_REPORT.md
B	AI Critique (297 từ)	ai/critique/AI_CRITIQUE.md
C	Nhật ký AI theo thời gian thực	ai/AI_log.md
D	Prompt gốc của từng bước	ai/prompts/
E	Test case dạng Excel (3 sheet + Summary)	testcases/23127060_HW06_testcases.xlsx
F	Git commit log	git-log/23127060_git_commit_log.txt
G	Bảng chứng header chống gian lận	ci/evidence/header_evidence.md

Tuyên bố sử dụng AI (đề bài mục 9)

I use AI tools for the following tasks.

Công cụ: **Claude Code** (`claude-opus-5`). Toàn bộ **14 lượt tương tác** được ghi lại **tự động ngay tại thời điểm xảy ra** bằng `scripts/ai_log.py` . Mỗi lượt lưu đầy đủ prompt gốc và tóm tắt output.

Các số liệu passed/failed **không** do AI ước lượng mà tính từ `newman/*.json` thật qua `summarize_newman.py` . Sơ đồ bộ sinh test là do **em tự vẽ**, không do AI sinh (đề bài mục 11).